

Start here: published orbit

User guide for the current **orbits.py** workflow. Updated September 26, 2026. Examples use WDS 16289+1825 and your **InputOrbit** folder.

1. Open Terminal and select your working folder

```
cd ~/Desktop/Orbits/InputOrbit
```

Keep historical input files and editable weights files in InputOrbit. The program is one folder above it, so commands use **../orbits.py**. Your desktop Orbits shortcut points to that program folder.

2. Show the published orbit

```
python3 ../orbits.py wds16289+1825_edit.txt --show
```

The historical text file supplies both the measurements and the published orbital elements. It must contain MEASURES: and ORBITAL ELEMENTS: sections. Only complete epoch/theta/rho measurements are used.

The program saves **wds16289+1825_edit_published_orbit.png** beside the input file. **--show** also opens the interactive plot. Without it, the plot is saved without a window. Use the plot toolbar for zoom, pan, reset, and Save.

3. Show the published orbit with WRMS

```
python3 ../orbits.py wds16289+1825_edit.txt \
  --weights wds16289+1825_edit_weights.csv --show
```

The orbital-elements panel shows **Published WRMS** in milliarcseconds (mas). This uses your selected weights without changing the published orbit. Without a weights file, WRMS is not displayed.

4. Save residuals and choose a plot name

```
python3 ../orbits.py wds16289+1825_edit.txt \
  --weights wds16289+1825_edit_weights.csv \
  --output WDS16289_plot.png \
  --residuals WDS16289_residuals.csv --show
```

Close the plot window to let this published-orbit command finish writing the residual CSV. The CSV includes observed and calculated theta/rho, residuals, and the assigned weight when **--weights** is supplied.

Copy the whole command block. A backslash at the end of a line continues the command; do not add spaces after it. Use two hyphens: **--weights**, not **-weights**. Quotes are needed around filenames containing spaces.

Create and edit weights

Create a new template

```
python3 ../orbits.py wds16289+1825_edit.txt \
  --create-weights
```

This creates **wds16289+1825_edit_weights.csv** beside the historical file. It lists only technique codes appearing in complete epoch/theta/rho pairs. Measurement counts also refer only to complete pairs.

--create-weights runs alone. Do not combine it with --show, --weights, --residuals, --output, --additional, or --optimize. It does not produce a plot or residuals.

Edit the CSV in Excel

Open the file in Excel, edit the **weight** column, and save as CSV. Keep the column headers and technique codes unchanged. The measurement_count column is informational and is not used to assign weights.

Column	Meaning
technique_code	Exact code used to match observations.
measurement_count	Number of complete historical pairs for that code.
weight	Relative nonnegative numeric weight for each pair.

New templates start at zero. A zero weight excludes that observation from optimization but leaves it visible in plots and residual exports. All-zero files may be loaded for review; they cannot be optimized. No automatic technique weights are assigned.

Every technique present in the data being fitted needs a weight, including codes from an optional additional-observation file. Missing, duplicate, negative, blank, or nonnumeric weights cause an error. Extra unused technique codes are reported.

Keep separate files for separate experiments

Use Excel Save As to make named alternatives, such as **weights_baseline.csv**, **weights_trial2.csv**, and **weights_no_J.csv**. Change the filename after --weights to choose the experiment.

If the template already exists, --create-weights stops without changing it. Rename or move the existing file before intentionally generating a fresh template. It does not update counts in an existing file.

Interpret WRMS consistently

$WRMS = \sqrt{\sum(w \times \text{positional residual squared}) / \sum(w)}$. The positional residual combines east and north errors in arcseconds; the display converts the result to mas. Zero-weight pairs do not contribute. If all matched weights are zero, the published display reports WRMS as unavailable.

Compare published and optimized WRMS using the **same weights**. Changing the weights changes the score, so the smallest WRMS across different weight files does not by itself identify the best orbit.

Optimize and compare experiments

Run an optimization

```
python3 ../orbits.py wds16289+1825_edit.txt \
  --weights wds16289+1825_edit_weights.csv \
  --optimize --show
```

Orbits starts from the published elements in the historical file and adjusts all seven orbital elements using weighted east/north least squares. Each run starts from that published orbit, not from the previous fitted result.

The plot shows the optimized orbit as a solid blue line and the published orbit as a dashed line. The elements panel shows the fitted values, **Published WRMS**, and **Optimized WRMS**. All complete observations remain visible.

Run another weights experiment

```
python3 ../orbits.py wds16289+1825_edit.txt \
  --weights weights_trial2.csv --optimize --show
```

First save `weights_trial2.csv` in InputOrbit. Omit `--show` if you only want the saved results. Do not use `--output` or `--residuals` with `--optimize`: optimization automatically creates all its outputs.

Find the results

Each run creates a new folder beside the historical file, named after the weights file plus a timestamp. For example: **weights_trial2_fit_YYYYMMDD_HHMMSS_fffff**. Earlier runs are preserved, even when you reuse the same weights file.

orbit_comparison.png - published and optimized orbits, fitted elements, and WRMS.

published_residuals.csv - residuals against the starting published orbit.

optimized_residuals.csv - residuals against the fitted orbit. Both CSVs include technique weights.

fit_report.json - both sets of elements, WRMS, weighted squared errors, convergence status, limits, and input details. Open in a text editor.

historical_input.txt and **weights_input.csv** - copies of the files used. An additional-input copy is saved when supplied.

For optimization, the residual CSVs and report are saved **before** the plot window opens.

What the fit does and does not establish

This is a local bounded fit from one starting orbit. It does not guarantee a global or unique solution and does not assign formal parameter uncertainties or an orbit grade. At least four positive-weight pairs at distinct epochs are required, but that minimum alone does not ensure a well-constrained orbit.

Keep the input data fixed when testing weights. Review changes in period and other elements, residuals across epochs, and convergence information alongside WRMS.

Options and troubleshooting

- show** - Open the interactive plot after saving it. Omit for file-only output.
- weights FILE.csv** - Load a selected weights CSV; show WRMS and add weights to residual exports.
- create-weights** - Create a zero-weight template only. Never overwrite an existing template.
- optimize** - Fit from the published orbit. Requires --weights. Automatically saves a new run folder.
- output FILE.png** - Choose a published-orbit plot filename; .pdf is also supported. Not for --optimize.
- residuals FILE.csv** - Export published-orbit residuals. Not for --optimize, which exports both sets automatically.
- additional FILE.csv** - Include additional observations. With --weights, their technique codes also need weights.
- help or -h** - Display the command-line options and exit.

Add an observation file

```
python3 ../orbits.py wds16289+1825_edit.txt \
  --additional added_observation.csv \
  --weights weights_trial2.csv --optimize --show
```

Supply your own added_observation.csv. Its supported columns include Date (decimal year), Theta_deg (degrees), Rho_arcsec (arcseconds), Aperture_meters, Reference, and Tech_code. Date, theta, and rho must be valid numbers. Additional records are not included in --create-weights templates; add their technique codes to your weights CSV if needed.

Common problems

- Cannot open ../orbits.py:** run `cd ~/Desktop/Orbits/InputOrbit` first. Use `pwd` to check the current folder and `ls` to list its files.
- Weights file already exists:** use that file, save a separate trial copy, or rename it before creating a fresh template.
- Missing technique weight:** add the exact reported code to the weights CSV with a numeric weight. Save as CSV, not Excel workbook format.
- No residual CSV yet:** close the published-orbit plot window so that command can finish.
- All weights zero:** enter positive values for measurements you intend to fit. Zero deliberately excludes measurements.
- Unrecognized argument:** use two plain hyphens for long options, such as --weights. Run `python3 ../orbits.py --help`.
- Plot will not open:** omit --show to save the image without opening an interactive graphics window.

Output and setup notes

Ordinary plot and residual filenames may be overwritten when reused; choose distinct names to retain them. Optimization folders and weights-template creation protect existing results. Python, NumPy, Matplotlib, and SciPy are installed for the current setup. The Run Orbits.command launcher opens a file chooser for a published plot; use Terminal for weights and optimization options.

Optimization methods: current fit

The current `--optimize` option uses **weighted nonlinear least squares**. It asks: which orbital elements best reproduce the measured positions, given the relative importance assigned by the selected weights file? The weights stay fixed during the fit.

What is adjusted

All seven elements vary: period P , semimajor axis a , inclination i , node angle Ω , periastron epoch T , eccentricity e , and argument of periastron ω . The initial values come from the published orbit in the historical file.

For each measurement epoch, Orbits solves Kepler's equation and projects the model orbit onto the sky. It converts the observed separation ρ and position angle θ to east and north coordinates: $\text{east} = \rho \times \sin(\theta)$, $\text{north} = \rho \times \cos(\theta)$. Both residual components are therefore in arcseconds, avoiding a direct mixture of degrees and arcseconds.

The quantity being minimized

$$S = \sum [w_i \times (dE_i^2 + dN_i^2)]$$

$$WRMS = \sqrt{ S / \sum(w_i) }$$

Here dE and dN are observed-minus-model east and north offsets, and w is the technique weight assigned to that pair. WRMS is reported in mas after multiplying its arcsecond value by 1,000. For a fixed data set and fixed weights, minimizing S also minimizes WRMS.

The solver multiplies residuals by square-root weights before squaring. Multiplying every weight by the same positive constant leaves the mathematical best fit and WRMS unchanged.

How the search proceeds

SciPy's **Trust Region Reflective (TRF)** least-squares solver makes successive parameter adjustments while respecting bounds. It uses the ordinary squared-error loss. Starting from the published orbit, it seeks a nearby minimum; it does not search every possible orbit. See the SciPy reference below.

Internally, Orbits uses log period, log semimajor axis, and phase at the median active epoch for scaling. Results are converted back to the familiar orbital elements.

The current search bounds are: P from 0.001 to 10,000,000 years; a from 0.00000001 to 100,000 arcseconds; i from 0 to 180 degrees; and e from 0 to 0.999999. These broad numerical limits are not scientific uncertainty intervals.

What convergence means

A successful solver status means its stopping rule was satisfied, not that the orbit is unique or physically definitive. The report records convergence, search bounds, and warnings about parameters at a bound or a rank-deficient Jacobian (parameters that cannot be independently constrained). No formal fitted-element uncertainties are calculated.

Method reference: [SciPy least_squares documentation](#). Program-specific settings above describe this version of Orbits.

Optimization methods: evaluating trials

How to interpret the weights

The current model applies the same weight to east and north errors and treats observations independently. Weights are relative coefficients, not measured error bars. For equal residuals, a pair weighted 10 contributes ten times as much squared error as a pair weighted 1. A zero-weight pair contributes nothing to the fit.

A technique represented by many observations has more aggregate weight than the same technique represented by only a few. However, its influence on an individual orbital element also depends on epoch and orbital geometry. A small number of old observations can constrain the period even when their weights are low.

Because residuals are squared, a large outlier can dominate despite a modest weight. Before lowering an entire technique, inspect the observation's epoch, reference, flags, theta, and rho. The current code does not automatically remove outliers or alter weights.

A controlled trial-and-error workflow

- 1. Preserve a baseline.** Keep the same historical input and save a baseline weights CSV. Each run already stores copies of its inputs.
- 2. Change one choice at a time.** For example, change one technique weight or one related family of weights. Save it under a new filename and record the reason.
- 3. Compare the whole result.** Check convergence, the seven elements, the orbit overlay, and residuals by epoch and technique. Look for parameter stability and systematic residual patterns, not just a smaller score.
- 4. Use the same scoring weights for a fair numerical comparison.** Published and fitted WRMS within one run use the same weights. Across runs with different weight files, WRMS measures different emphases. The current program does not automatically rescore every trial with a shared reference weighting.

Other methods we could add later

Multiple starting orbits. Repeat the local fit from several plausible starting elements. Agreement would provide evidence of stability; different good fits would reveal ambiguity. This is not currently implemented.

Robust least squares. A loss such as soft-L1 or Huber reduces the effect of large residuals compared with ordinary squaring. It requires a meaningful residual scale and decisions about how to treat paired east/north errors. SciPy supports these losses, but Orbits currently uses ordinary least squares.

A broader search, followed by local refinement. Differential evolution is a population-based global-search method that could explore specified parameter ranges before TRF refines a candidate. It needs defensible bounds and more computation; it does not guarantee that a unique physical solution exists. It is not currently implemented.

Uncertainty and sensitivity analysis. Repeated fits to resampled observations or perturbed measurements could assess parameter stability. Such estimates require assumptions about measurement errors and dependence between observations; they are not part of the current output.

References: [SciPy least_squares \(ordinary and robust losses\)](#); [SciPy differential_evolution](#). Future methods above are options for development, not available command-line switches.